

Agent GitHub 使用场景

1. 新人clone整个团队的context。

小林周一入职支付组。按老办法，他得花一周翻代码、翻 wiki、挨个问人，才能大概搞清楚这套系统。现在他在自己机器上跑 `agentgit clone payments`，本地 agent 就 checkout 到支付组这条分支，直接带上了整个团队攒下来的context。

操作：**clone、checkout**

2. 本地几个 Agent 之间同步。

小林这几天都在退款这块打转。第一天他让 Agent 把退款流程从头读了一遍，搞清楚走哪几个服务、哪几处有历史坑。可会话一关，这份理解就没了，第二天开新会话修另一个退款 bug，Agent 又得从零把同样的代码读一遍。（`--resume` 可以继续昨天的session，但是这样就只能顺着那一条session接下去）。所以小林用 `agentgit`，往后每次开新session，都先 checkout 这条分支。

操作：**commit、checkout**

3. 跨人、跨时区接力。

Alice 在旧金山，花一下午查清楚了线上延迟变高的原因，是因为 `OrderService.list` 里有个 N+1 查询，某次改动引进来的。她下班前 `agentgit push` 把这条分支推上去。在北京的小林第二天早上接着做，`agentgit pull alice/latency`，他的新 Agent 一开工就带着 Alice 的agent 结论和经验。

操作：**push、pull、checkout**

4. fork 出来尝试新方向。

小林想按一个新思路重构退款流程。他不在共享分支上直接改，而是 `agentgit fork payments` 拉出一条自己的分支，让 Agent 在这条 fork 上去试。方案跑通了，他再提个 merge 把有用的结论合回去；没跑通就直接把这条 fork 丢掉。

cc也有fork，但是非常难用，以及没办法merge回去

操作：**fork**

5. merge conflict。

Carol 做 review，要把 Bob 的实现上下文 merge 进来。 `agentgit merge bob/refund` 跑出一个冲突，Bob 的上下文说接口字段叫 `user_id`（依据 `models/user.ts:12`），而全组共享的 api-agent 里记的是 `uid`（依据一份很旧的文档）。AgentGit 可以把两条都列出来，带着各自的证据，交给 reviewer（人类/agent）判断。

操作：**merge、diff**

6. Agent 跨 Project 共用。

支付组的前端和后端是两个 Project，但是都共用同一个 api-agent。后端的 Agent 干活时发现接口新加了个限流响应头，`commit` 进 api-agent。前端开发第二天可以直接 `pull` api-agent。

操作：**commit、pull**

7. reset

有次 Bob 从一个没验证过的分支 merge 了一批关于鉴权流程的结论，里面有几条信息是错的，之后 Agent 给的建议就开始跑偏。`agentgit log` 之后发现问题出在那次 merge。Bob 可以直接 `agentgithub reset` 回到 merge 之前那个 commit，把之后的改动全部丢掉，回到干净状态。

操作：**log、reset**

8. push 之前先扫一遍密钥。

小林查一个连数据库的问题时，让 Agent `cat` 了一下 `.env`，又跑了条命令把 connection string 打了出来。这段东西现在就躺在他 Agent 的 context 里，一旦他 `push` 这条分支、同事 `clone` 下来，这个密码就跟着 context 发到每个人机器上。所以必须在 commit、push、merge 之前先扫一遍。

commit hook?